

DSC 40B: THEORETICAL FOUNDATIONS OF DATA SCIENCE II

BINARY SEARCH AND RECURRENCES

ON UNSORTED DATA, LINEAR SEARCH IS OPTIMAL

Worst case for `linear_search` is $\Theta(n)$. You have to look at every element before you can rule the target out.

Sorted input is a different problem, and the $\Theta(n)$ argument no longer applies. We also have no tool for the runtime of a function that calls itself.

Two jobs in this lecture: design an algorithm that exploits sortedness, then build the machinery that times it.

SEARCHING SORTED DATA

Why the input moves the lower bound, and the algorithm that reaches it.

THE INPUT CHANGES THE PROBLEM

UNSORTED ARRAY

Given arr and target t, return the index of t.

Lower bound: $\Omega(n)$. Skip any element and the adversary hides t there.

SORTED ARRAY

Same request, stronger base.

One comparison now rules out an entire side. Lower bound drops to $\Omega(\log n)$.

A lower bound is a claim about every algorithm for the problem, not about one implementation. It says what no one can beat.

ONE COMPARISON, HALF THE ARRAY GONE

- | | | |
|----|--------------------------------|---|
| 01 | Compare at the middle | Look at the element in the center of the active range. |
| 02 | Read the direction | If t is smaller, it can only lie to the left. If larger, only to the right. |
| 03 | Discard the other half | Sortedness rules that half out. No comparisons are spent on it. |
| 04 | Recurse on what remains | Same procedure, half the input. Stop when the range is empty. |

SLIGHT DETOUR: INTEGER DIVISION IS FLOOR DIVISION

DEFINITION

The floor of a real number x , written $\lfloor x \rfloor$, is the largest integer that is $\leq x$. The ceiling, written $\lceil x \rceil$, is the smallest integer that is $\geq x$.

FLOOR

$$\lfloor 3.14 \rfloor = 3$$

$$\lfloor -4.5 \rfloor = -5$$

$$\lfloor 10 \rfloor = 10$$

CEILING

$$\lceil 3.14 \rceil = 4$$

$$\lceil -4.5 \rceil = -4$$

$$\lceil 10 \rceil = 10$$

IN PYTHON

// is floor division.

$(\text{start} + \text{stop}) // 2$ is $\lfloor (\text{start} + \text{stop}) / 2 \rfloor$.

Rounding down is fine. The range still shrinks.

Floors will show up again when we count how many times n can be halved before it reaches 1.

BINARY SEARCH, IN FULL

```
def binary_search(arr, target, start=0, stop=None):
    if stop is None:
        stop = len(arr)

    if start >= stop:
        return None

    middle = (start + stop) // 2

    if arr[middle] == target:
        return middle
    elif arr[middle] < target:
        return binary_search(arr, target, middle + 1, stop)
    else:
        return binary_search(arr, target, start, middle)
```

stop defaults to None, not to len(arr). Default arguments are evaluated once when the function is defined, and arr does not exist yet at that point.

ONE RUN WITH TARGET: 34

value	-20	-13	3	6	10	12	15	34	56	76	89
index	0	1	2	3	4	5	6	7	8	9	10

CALL	MIDDLE	ARR[MIDDLE]	ACTION
binary_search(0, 11)	5	12	12 < 34. Search right. Recurse on (6, 11).
binary_search(6, 11)	8	56	56 > 34. Search left. Recurse on (6, 8).
binary_search(6, 8)	7	34	Match. Return 7.

How many passes for Linear Search?

RECURSION IS CHECKED IN THREE PLACES

01 The base case is correct

The smallest input where the answer is obvious. Verify it directly, by hand.

02 Every call is strictly smaller

If the input does not shrink, the base case is never reached and the recursion never ends.

03 The combination is correct

Assume the recursive calls return the right answer. Show the whole function then does too.

Check 3 is the leap of faith. It is induction: check 1 is the base case, check 3 is the inductive step, and check 2 guarantees you reach the base case.

BOTH BASE CASES HOLD

N = 0

len(arr) is empty, so start $\geq$ stop and the function returns None. There is nothing in the range to find. Correct.

N = 1

One element. If it matches target, its index (0) is returned. If not, the call recurses on a range of size 0, which returns None. Correct.

TERMINATION

Each recursive call runs on at most $\lfloor n/2 \rfloor$ elements, which is strictly smaller than n for every $n \geq 1$. Check 2 holds, so the base case is always reached.

Correctness is settled. The runtime is not, and no loop counter will settle it.

RECURRENCE RELATIONS

Timing a function that calls itself, then reducing that description to a closed form.

NO LOOP MEANS NOTHING TO COUNT

THE WORK OUTSIDE THE CALL

Compute middle, one comparison, one return.

Fixed cost. It does not depend on n .

$\Theta(1)$.

THE SIZE OF THE CALL

The range is cut in half before recursing.

One call, on $n/2$ elements.

Its cost is $T(n/2)$ by definition.

PUT THEM TOGETHER

Constant work plus one half-size call.

That is the entire body.

Nothing else executes.

$$T(n) = T(n/2) + \Theta(1)$$

A recurrence relation. T is defined in terms of itself, so it is not yet an answer.

Best case is $\Theta(1)$: the target sits at the first middle. This recurrence describes the worst case.

TWO SIMPLIFICATIONS, BOTH SAFE

DROP THE THETA

Solve $T(n) = T(n/2) + 1$ instead. The constant is absorbed by Θ at the end, so carrying it through the algebra buys nothing.

ASSUME N IS A POWER OF 2

Then $n/2$ is always an integer and no floors appear. The growth rate is unchanged, since $\lfloor n/2 \rfloor$ differs from $n/2$ by less than 1.

Both are shortcuts for the algebra, not changes to the algorithm. The final answer must still be stated in Θ .

UNROLLING IS A FOUR-STEPS, AGAIN..

- | | | |
|----|--|---|
| 01 | Unroll a few times | Substitute the recurrence into itself. Three levels is usually enough to see the pattern. |
| 02 | Write the kth line | Generalize. Express $T(n)$ after k substitutions, with k as a variable. |
| 03 | Solve for k at the base case | Find the k that drives the argument down to the base case. |
| 04 | Substitute k back in | Plug that k into the k th line. Read the growth rate off the result. |

A WARM-UP THAT IS NOT AN ALGORITHM

Day 1 pays \$3. Every day after that, the pay doubles. $S(1) = 3$, $S(n) = 2 \cdot S(n-1)$

$$S(2) = 2 \cdot S(1) = 2 \cdot 3$$

$$S(3) = 2 \cdot S(2) = 2 \cdot 2 \cdot S(1)$$

$$S(n) = 2 \cdot S(n-1)$$

$$= 2 \cdot 2 \cdot S(n-2) = \dots$$

kth line:

base case:

substitute:

$$S(n) = 3 \cdot 2^{n-1} = \theta(2^n)$$

Exact solution first, asymptotic solution second.

Subtracting from the argument gives an exponential. Dividing it gives a logarithm. That contrast is the reason this example is here.

UNROLL THE BINARY SEARCH RECURRENCE

$$\begin{array}{llll} T(n) = T(n/2) + 1 & & & k = 1 \\ = [T(n/4) + 1] + 1 & = & T(n/4) + 2 & k = 2 \\ = [T(n/8) + 1] + 2 & = & T(n/8) + 3 & k = 3 \end{array}$$

$$\text{kth line: } T(n) = T(n / 2^k) + k \quad \text{step 2}$$

THE BASE CASE FIXES k , AND k IS THE ANSWER

step 3: $T(1)$ is reached when $n / 2^k = 1$

step 4: $T(n) = T(1) + \log_2 n$
 $= 1 + \log_2 n$

worst case $T(n) = \Theta(\log n)$

best case $T(n) = \Theta(1)$

The base of the logarithm is dropped. $\log_2 n = (\log n) / (\log 2)$, and $1 / \log 2$ is a constant, so every base lands in the same Θ class. Write $\Theta(\log n)$.

THE DIFFERENCE IS NOT A CONSTANT FACTOR

Every grain of sand on Earth, roughly 10^{19} of them, numbered and sorted. One basic operation takes 1 nanosecond.

ALGORITHM	COMPARISONS, WORST CASE	WALL CLOCK
linear_search	about 10^{19}	about 317 years
binary_search	about 63	about 60 nanoseconds

$\log_2(10^{19}) \approx 63$. Doubling the dataset adds exactly one comparison. That is what logarithmic growth means in practice.

SORTEDNESS IS NOT FREE

THE COST

The array has to be sorted first, and sorting is $\Omega(n)$. For a single query on unsorted data, `linear_search` wins. Sort plus binary search is strictly more work.

WHEN IT IS USEFUL TO SORT

Sort once, query many times. The sort is paid once and amortizes across queries, and every query after it costs $\Theta(\log n)$.

OPTIMALITY

The tight lower bound for searching a sorted array is $\Omega(\log n)$. Binary search matches it, so its worst-case time complexity is optimal. No algorithm does better on this problem.

Next lecture: how sorting itself is done, and what it costs.

YOUR TURN

```
def mystery(arr, start=0, stop=None):  
    if stop is None:  
        stop = len(arr)  
  
    if stop - start <= 1:  
        return arr[start]  
  
    middle = (start + stop) // 2  
    left = mystery(arr, start, middle)  
    right = mystery(arr, middle, stop)  
    return max(left, right)
```

SOLVE

Write the recurrence for $T(n)$.

Solve it by unrolling. Show all four steps.

State the worst-case time complexity in Θ .

Assume n is a power of 2.

The structure is close to binary search. One detail is different, and it changes the answer completely.

SOLUTION

$$\begin{aligned}T(n) &= 2 \cdot T(n/2) + 1 & k = 1 \\&= 4 \cdot T(n/4) + 2 + 1 & k = 2 \\&= 8 \cdot T(n/8) + 4 + 2 + 1 & k = 3\end{aligned}$$

kth line: $T(n) =$ step 2
base case: step 3
substitute: step 4

$$T(n) = \Theta(n)$$

Two half-size calls, not one. The recursion visits every element.

Binary search discards half the input at each level. This function does not. The number of leaves dominates, not the depth of the tree.